ENTERPRISE HIGH-CONCURRENCY BLUEPRINT

ShagunShopping
Zero-Oversell High-Scale Architecture

Redis, Apache Kafka, Docker & Kubernetes

A battle-tested architectural specification solving the high-concurrency race condition on limited inventory (2 customers purchasing the last item at the exact same millisecond), complete with event-driven pipelines, containerization, and auto-scaling Kubernetes cluster manifests.

0%	< 2ms	Kafka	2 - 10
OVERSELL PROBABILITY	STOCK LOCK LATENCY	EVENT BACKBONE	K8S HPA PODS

CORE TECHNOLOGIES	CONCURRENCY MODEL	DEPLOYMENT	PRODUCTION TARGET
Redis 7.2, Kafka 3.7, K8s	Atomic Lua Leased Holds	Docker Compose + K8s	20,000+ Concurrent Users

1. The Critical Concurrency Problem: Racing for the Last Item

In e-commerce, the most dangerous failure mode occurs during high-traffic moments (festive offers, flash sales, Instagram promotions) when **Product X has exactly 1 unit left in stock**, and **Customer A and Customer B click "Pay" at the exact same millisecond**.

Failure Mode in Naive Architectures (Double-Charge & Oversell)

- Both Customer A and Customer B read `stock = 1` from MongoDB.
- Both get redirected to Razorpay and authenticate payment on their phones.
- Customer A pays at 12:00:01.050 ➔ Stock decrements to 0.
- Customer B pays at 12:00:01.075 ➔ Bank deducts ₹1,499 from Customer B!
- Server attempts to fulfill Customer B ➔ **OUT OF STOCK!**
- **Result:** Customer B's money is debited, order is cancelled, customer is furious, and merchant must issue manual refunds.

Failure Mode in Naive Pre-Deductions (Stock Hostage / Ghost Stock)

- Customer A clicks checkout ➔ Server decrements stock immediately in DB (stock = 0).
- Customer B arrives ➔ Sees "Out of Stock" and leaves the website.
- Customer A gets a phone call, battery dies, or abandons the Razorpay modal.
- **Result:** Stock remains at 0 in the database indefinitely. The merchant loses the sale to Customer B, and the inventory is held hostage.

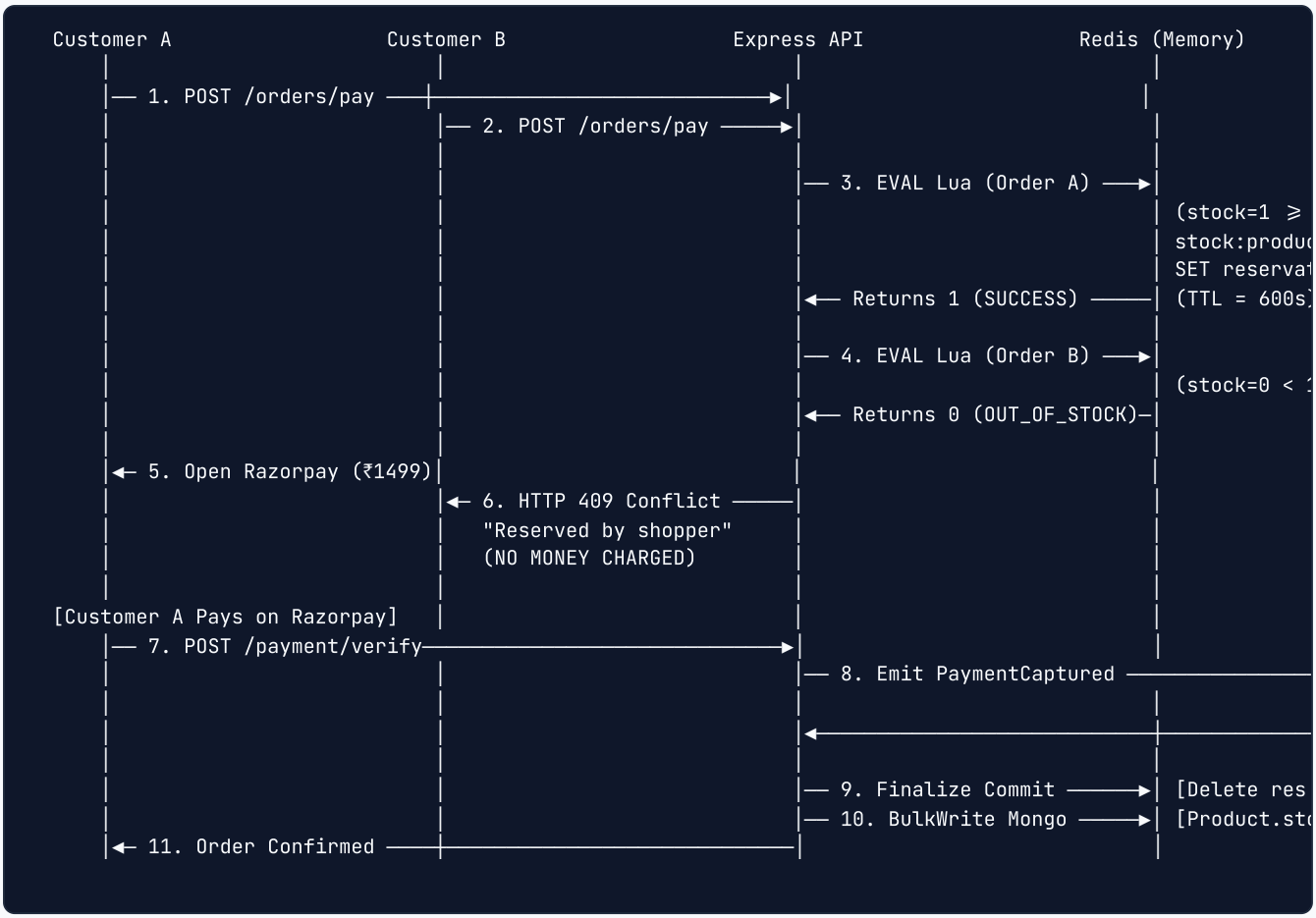
The Enterprise Solution: Atomic Two-Phase Leased Reservation with Redis & Kafka

The Three Pillars of Zero-Oversell Consistency

1. **Phase 1: Atomic Soft-Lock in Redis via Single-Threaded Lua Script:** The moment Customer A enters payment, an atomic Lua script decrements the Redis stock counter and creates an expiring reservation key (TTL = 10 minutes). Redis executes Lua scripts atomically in a single thread—Customer B's script runs 1 millisecond later, sees 0 stock, and is cleanly rejected with HTTP 409 *before any payment gateway is ever invoked*. Customer B is NEVER charged.
2. **Phase 2: Asynchronous Event via Apache Kafka on Payment Capture:** Once Customer A completes payment on Razorpay, a `PAYMENT_CAPTURED` event is emitted to Kafka topic `shagunshopping.payment.events`. The inventory consumer finalizes the MongoDB deduction and clears the temporary reservation.
3. **Phase 3: Automatic TTL Compensating Release:** If Customer A closes the tab or fails payment, the Redis reservation key expires after 10 minutes, automatically returning the unit to available stock without locking the primary database.

2. Distributed Concurrency Sequence Diagram

SEQUENCE FLOW: CUSTOMER A VS CUSTOMER B RACING FOR THE LAST UNIT (STOCK = 1)



2.1 The Atomic Lua Script Implementation

Implemented in `backend-shagunshopping/src/services/inventoryLockService.js`:

```
-- Atomic Lua Script executed directly inside Redis
local stock_key = KEYS[1]
local reservation_key = KEYS[2]
local req_qty = tonumber(ARGV[1])
local ttl_seconds = tonumber(ARGV[2])

local current_stock = tonumber(redis.call('get', stock_key) or '-1')

if current_stock == -1 then
    return -1 -- Signal to application: seed stock from MongoDB
end

if current_stock >= req_qty then
    redis.call('decrby', stock_key, req_qty)
    redis.call('set', reservation_key, req_qty, 'EX', ttl_seconds)
    return 1 -- SUCCESS: Reserved exclusively for this customer
else
    return 0 -- FAILED: Out of stock or already reserved
end
```

3. Apache Kafka Event-Driven Backbone

Rather than handling order confirmation, warehouse pick-lists, invoice PDF generation, customer SMS, and email alerts synchronously within the API request thread, the platform adopts an **Event-Driven Architecture (EDA)** via Apache Kafka.

3.1 Kafka Topic Topology & Event Routing

TOPIC NAME	PARTITION KEY	PRODUCED BY	CONSUMED BY	PAYLOAD SUMMARY
`shagunshopping.order.events`	`orderId`	Order Controller	Inventory Service, Analytics, CRM	`ORDER_CREATED`, `ORDER_CANCELLED`, line items, user details, total price.
`shagunshopping.payment.events`	`orderId`	Payment Controller / Webhook	Inventory Consumer, Accounting, Mailer	`PAYMENT_CAPTURED`, `PAYMENT_FAILED`, Razorpay payment ID, signature digest.
`shagunshopping.inventory.events`	`productId`	Inventory Lock Service	Storefront WebSocket Hub, Catalog Cache	`STOCK_RESERVED`, `STOCK_RELEASED`, `STOCK_EXHAUSTED`. Triggers live UI updates.
`shagunshopping.notification.events`	`userId`	Order & Auth Controllers	Notification Worker Pods	`SEND_OTP`, `SEND_ORDER_CONFIRMATION`, `SEND_SHIPPED_SMS`, WhatsApp OTP.

3.2 Consumer Group Scaling & Fault Tolerance

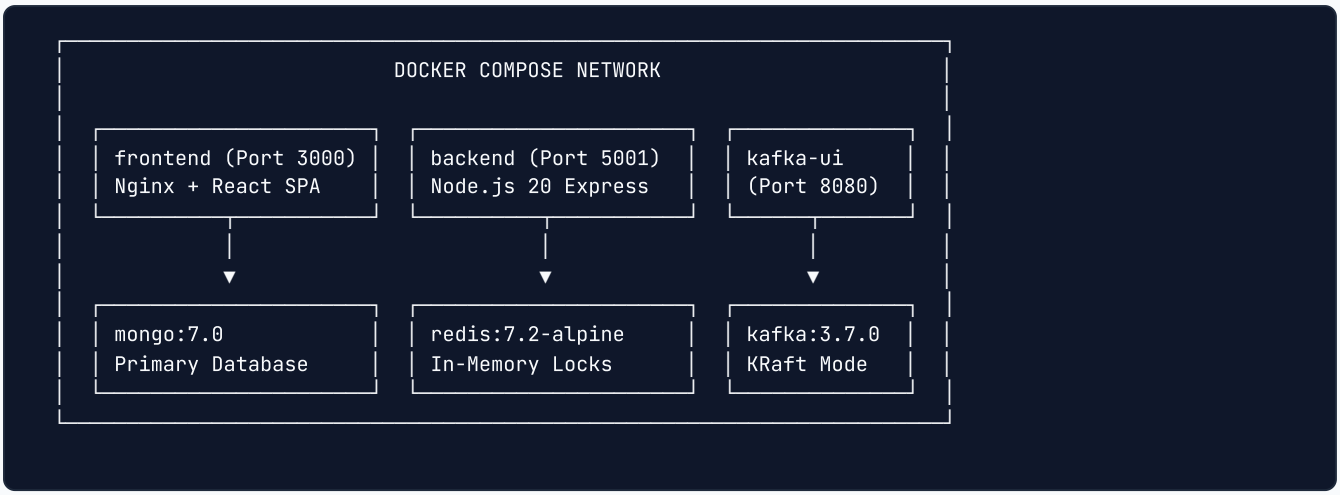
- **Order Preservation via Partition Keys:** By setting `orderId` as the partition key, Kafka guarantees that all events for a specific order (Created ➔ Paid ➔ Shipped ➔ Delivered) arrive at the consumer in strict chronological order.
- **Consumer Group Load Balancing:** The `shagunshopping-inventory-group` scales horizontally across multiple Kubernetes pods. If Pod 1 crashes, Kafka triggers an automatic consumer group rebalance to Pod 2 without dropped messages.
- **Dead Letter Queue (DLQ):** If an event fails processing after 5 retry attempts (e.g. malformed address format), it is routed to `shagunshopping.dlq` with the full stack trace for administrator inspection.

4. Containerization & Kubernetes Cluster Architecture

4.1 Local Multi-Container Development (docker-compose.yml)

The entire enterprise stack boots locally with a single command: `docker compose up -d`.

DOCKER COMPOSE SERVICE MAP



4.2 Production Kubernetes (K8s) Cluster Specification

Structured into the `shagunshopping` namespace with declarative manifests inside the `k8s/` directory:

MANIFEST FILE	RESOURCE KIND	PRODUCTION SPECIFICATION
`k8s/namespace.yaml`	Namespace	Isolates all workloads within `shagunshopping`.
`k8s/config-and-secrets.yaml`	ConfigMap & Secret	Stores environment configurations, MongoDB connection string, and cryptographic secrets.
`k8s/redis.yaml`	Deployment & Service	Redis 7.2 Alpine with AOF persistence, liveness/readiness probes, and 512Mi memory limit.
`k8s/kafka.yaml`	Deployment & Service	Apache Kafka 3.7.0 in modern KRaft mode (no ZooKeeper dependency), 3-partition setup.
`k8s/backend.yaml`	Deployment & Service	Express API with rolling updates (maxSurge: 1, maxUnavailable: 0), health probes on `/api/health`.
`k8s/frontend-ingress-hpa.yaml`	Frontend, Ingress, HPA	Nginx React SPA, Ingress with Let's Encrypt TLS, and Horizontal Pod Autoscaler (HPA: 2 to 10 pods).

4.3 Horizontal Pod Autoscaling (HPA) Policy

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: shagunshopping-backend-hpa
  namespace: shagunshopping
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: shagunshopping-backend
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
```

5. Step-by-Step Implementation Checklist

1. Distributed Inventory Locking

- `ioredis` installed in backend.
- `inventoryLockService.js` with atomic Lua reservation script.
- Pre-payment lock call added to `orderController.js`.
- Commit on payment wired into `paymentController.js`.
- Compensating rollback on order cancellation.

2. Apache Kafka Event Pipeline

- `kafkajs` installed in backend.
- `kafka.js` configuration for KRaft broker connections.
- `eventProducer.js` emitting `ORDER_CREATED`, `PAYMENT_CAPTURED`.
- `inventoryConsumer.js` consumer group for asynchronous inventory management.

3. Docker Orchestration

- Multi-stage production `Dockerfile` for backend.
- Multi-stage production `Dockerfile` with Nginx for frontend.
- `docker-compose.yml` running API, SPA, Mongo, Redis, Kafka, and Kafka UI.

4. Kubernetes Production Deployment

- Complete declarative manifests in `k8s/`.
- Horizontal Pod Autoscaler (HPA) scaling 2-10 pods.
- Nginx Ingress controller with Let's Encrypt automated SSL.
- Liveness and readiness health probes on all pods.